Redis Replication, Sentinel and Cluster

high availability, and scaling beyond one machine

Phase 3 videos on YouTube

In one line: replicas copy the primary asynchronously and can serve reads; Sentinel watches the primary and promotes a replica when a quorum agrees it's down; Redis Cluster splits keys into 16,384 hash slots spread over several primaries, clients route by slot and follow MOVED redirects, and writes not yet replicated can be lost on failover.

1 One Redis is a single point of failure

Your Redis server holds every user session. One night it dies: everyone is logged out, and the app can't serve anyone until it's back. One node is a single point of failure, and it also caps your memory and throughput. Replication, Sentinel and Cluster solve those problems.

2 Replication

A **primary** accepts writes, and one or more **replicas** copy its data. Replication is **asynchronous**: the primary acknowledges a write to the client before replicas receive it. Replicas can serve reads and stand ready to take over. `WAIT 1 100` blocks until at least one replica confirms (or 100 ms pass), but by default Redis favours speed over durability.

3 Sentinel: automatic failover

Who promotes a replica when the primary dies? **Redis Sentinel**: a group of Sentinel processes, usually **three or more** on separate machines, monitors the primary. When a **quorum** agrees it's down, the Sentinels elect a leader that promotes the best replica (least lag), repoints the other replicas, and tells clients the new primary's address. Clients ask Sentinel where the primary is instead of hard-coding it.

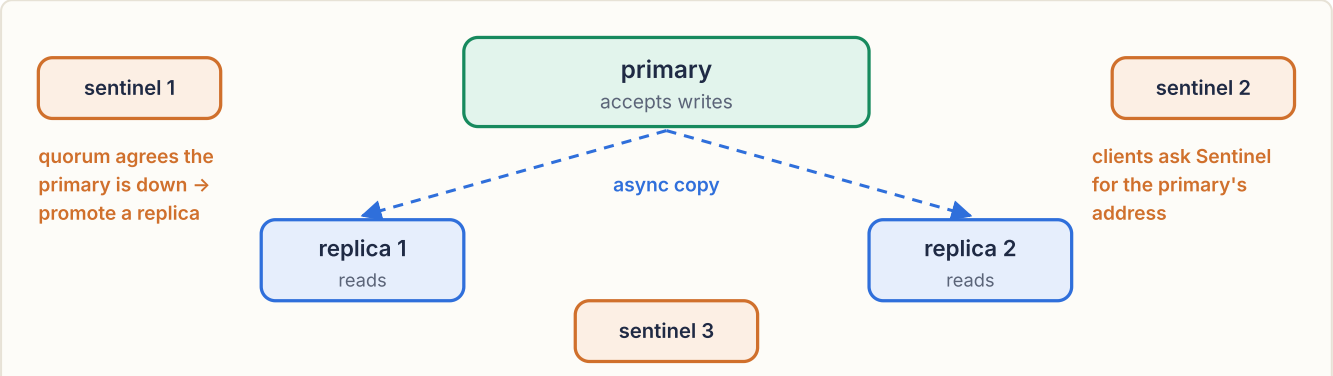


Figure 1. Replication with Sentinel: three Sentinels watch the primary; on failure a quorum promotes a replica and clients follow.

4 Redis Cluster: scaling out

Sentinel gives high availability, but all data still fits on one primary. To scale beyond one machine, use **Redis Cluster**. The key space is split into **16,384 hash slots**: a key's slot is `CRC16(key) mod 16384`. Each primary owns a range of slots, and each has its own replicas, which the cluster promotes automatically.

Clients learn the **slot map** and send each command straight to the right node. If a slot has moved, the node replies with a **MOVED** redirect (or **ASK** during a migration), and the client updates its map. **Resharding** moves slots between nodes while the cluster keeps serving; adding a node means migrating some slots to it.

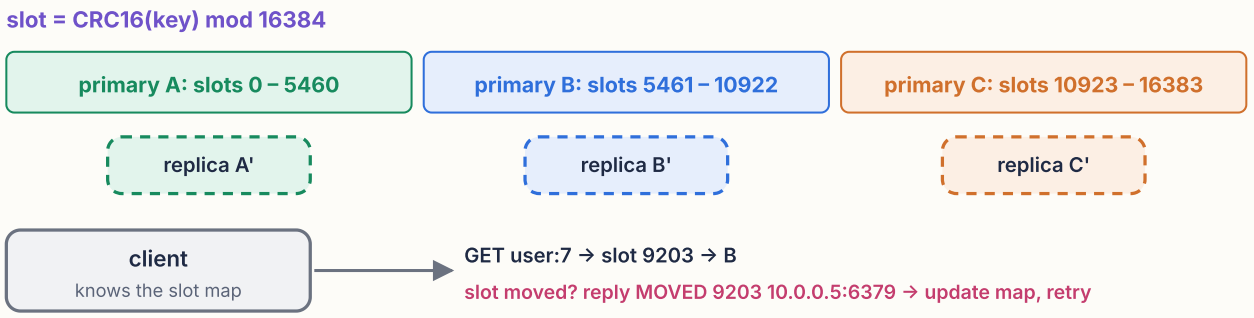


Figure 2. Redis Cluster: three primaries own slot ranges; clients route by slot and follow MOVED when slots move.

5 Multi-key operations and hash tags

Commands that touch several keys (`MGET`, `SUNION`), transactions and Lua scripts only work when **all keys are in the same slot**. **Hash tags** solve this: only the part of the key inside curly braces is hashed, so `user:{42}:cart` and `user:{42}:profile` land in the same slot. Use them deliberately: put everything under one tag and you create a hot slot.

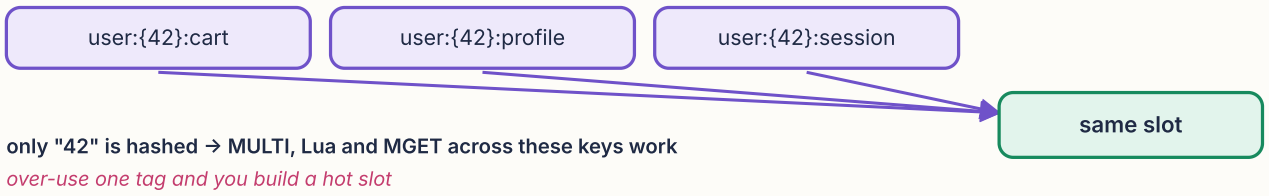


Figure 3. Hash tags: the braces decide the slot, so related keys stay together.

WORKED EXAMPLE: A SESSION CLUSTER

A six-node Redis Cluster stores sessions: three primaries, each owning about **5,461 slots**, and one replica per primary in another availability zone. If a primary fails, its replica is promoted automatically within seconds (after `cluster-node-timeout`, e.g. 15 s). But because replication is asynchronous, writes acknowledged by the failed primary but not yet copied are **lost**: a few users may lose a recent session update, or need to log in again.

COMMON MISTAKES

- Two Sentinels (no majority possible when one fails), or all Sentinels on the same machine as the primary.
- Clients hard-coding the primary's address instead of asking Sentinel, or not handling MOVED.
- Multi-key Lua scripts without hash tags: `CROSSSLOT` errors in production.
- Assuming failover loses nothing: asynchronous replication always has a window.

6 Key terms

Replica: a copy of the primary, updated asynchronously

Sentinel: processes that monitor and fail over a primary

WAIT: block until N replicas confirm a write

Quorum: how many Sentinels must agree it's down

Hash slot: one of 16,384 buckets of the key space

Resharding: moving slots between nodes while online

MOVED / ASK: redirects to the node that owns a slot

Hash tag: {part} of a key that alone decides the slot

7 Interview questions

Q1. How does Redis Cluster distribute keys? BASIC

Keys are hashed with CRC16 into 16,384 hash slots, slots are spread across primaries, clients route by slot and follow MOVED redirects, and hash tags keep related keys in one slot.

Q2. What data can be lost on failover? BASIC

Writes the primary acknowledged but hadn't yet replicated, because replication is asynchronous; WAIT and AOF reduce but don't eliminate that risk.

Q3. Sentinel or Cluster? BASIC

Sentinel when the data fits on one machine and you need high availability; Cluster when you need more memory or throughput than one primary, with failover built in.

Q4. Sessions must survive a node failure. How do you set up Redis? SCENARIO

Replicas in another zone with automatic failover (Sentinel or Cluster), AOF everysec on primaries, WAIT 1 for critical writes if the latency is acceptable, and clients that reconnect to the new primary.

Q5. Why does a Lua script that touches user:42:cart and user:42:profile fail in Cluster? TRICKY

The keys hash to different slots, possibly on different nodes, so the script can't run atomically; rename them with a hash tag like user:{42}:cart so they share a slot.

Q6. What is split brain in Redis, and how is it limited? TRICKY

During a partition the old primary may keep accepting writes while a replica is promoted elsewhere; those writes are lost when it rejoins. min-replicas-to-write makes an isolated primary stop accepting writes, limiting the damage.

Next: SD-F-015 · Memcached vs Redis backendroadmap.com